

Translated

From: e-maxx.ru

Solving assignment problem using min-cost-flow

The **assignment problem** has two equivalent statements:

- Given a square matrix $A[1..N, 1..N]$, you need to select N elements in it so that exactly one element is selected in each row and column, and the sum of the values of these elements is the smallest.
- There are N orders and N machines. The cost of manufacturing on each machine is known for each order. Only one order can be performed on each machine. It is required to assign all orders to the machines so that the total cost is minimized.

Here we will consider the solution of the problem based on the algorithm for finding the [minimum cost flow \(min-cost-flow\)](#), solving the assignment problem in $\mathcal{O}(N^3)$.

Description

Let's build a bipartite network: there is a source SS , a drain TT , in the first part there are N vertices (corresponding to rows of the matrix, or orders), in the second there are also N vertices (corresponding to the columns of the matrix, or machines). Between each vertex i of the first set and each vertex j of the second set, we draw an edge with bandwidth 1 and cost A_{ij} . From the source SS we draw edges to all vertices i of the first set with bandwidth 1 and cost 0. We draw an edge with bandwidth 1 and cost 0 from each vertex of the second set j to the drain TT .

We find in the resulting network the maximum flow of the minimum cost. Obviously, the value of the flow will be N . Further, for each vertex i of the first segment there is exactly one vertex j of the second segment, such that the flow $F_{ij} = 1$. Finally, this is a one-to-one correspondence between the vertices of the first segment and the vertices of the second part, which is the solution to the problem (since the found flow has a minimal cost, then the sum of the costs of the selected edges will be the lowest possible, which is the optimality criterion).

The complexity of this solution of the assignment problem depends on the algorithm by which the search for the maximum flow of the minimum cost is performed. The complexity will be $\mathcal{O}(N^3)$ using [Dijkstra](#) or $\mathcal{O}(N^4)$ using [Bellman-Ford](#). This is due to the fact that the flow is of size $\mathcal{O}(N)$ and each iteration of Dijkstra algorithm can be performed in $\mathcal{O}(N^2)$, while it is $\mathcal{O}(N^3)$ for Bellman-Ford.